รายงานโครงการ: การเรียนรู้แบบเสริมกำลังให้หุ่นมนุษย์ตัดสินใจกู้สมดุลหรือยอมล้มเพื่อลดความเสียหาย

Oct 11, 2026 · @Stang

บทคัดย่อ

หุ่นมนุษย์จำลองที่ถูกฝึกด้วย PPO เรียนรู้ได้ว่าแรงผลักครั้งใดควรกู้และครั้งใดควรยอมล้มเพื่อลดความเสียหาย โดย policy เดียวผ่านเกณฑ์ที่กำหนดไว้ล่วงหน้าครบทั้ง 3 seeds รายงานนี้สรุปงานขั้น S1 และ S2 ของโครงการ RL Fall และเสนอคำขอทรัพยากร LANTA สำหรับขั้น S3

งานใช้หุ่นผู้หญิงอ้างอิง 1.60 m / 60 kg บน MuJoCo (โครง SMPL 24 ท่อน สัดส่วนจาก de Leva 1996 และ ANSUR II) ฝึกแบบ curriculum สองขั้น ขั้น S1 ให้ยืนสบายแบบคนจริงและรอดแรงผลักด้วยการก้าวเท้าตอบสนอง ขั้น S2 fine-tune ต่อด้วยโทษ damage ถ่วงน้ำหนัก (ศีรษะ 5, ลำตัว 2, แขนและขา 1) บนแรงผลักที่กู้ได้และกู้ไม่ได้อย่างละครึ่ง

ผล S1 (M1): อัตรารอดเฉลี่ยที่ impulse 0.10 เท่ากับ 62% (ท่านิ่ง 0%) ท่ายืนก่อนผลักอยู่ในเกณฑ์ทุก seed และอัตราการก้าวเพิ่มตามแรงผลัก ผล S2 (M2) ที่ 192 episodes ต่อกลุ่ม: อัตรารอดบนแรงที่กู้ได้ 88.5–96.9% ไม่ต่ำกว่า S1 ทุก seed, damage บนแรงที่กู้ไม่ได้ต่ำกว่า S1 9–21% และต่ำกว่าการล้มแบบปล่อยตัว 13–28% และอัตรายอมล้มผิดจังหวะ (false give-up) 2.7–9.8%

ผลนี้ได้มาหลังลองตั้งค่า S2 ทั้งหมด 10 ชุด (ไม่ผ่าน 9 ชุดก่อนหน้า) ชุดที่ผ่านเปลี่ยน reward ของแรงผลักที่กู้ไม่ได้ให้เหลือแค่โทษ damage และผ่าน 3/3 เฉพาะที่ชุดประเมิน 192 episodes (ที่ 48 episodes เดิมผ่าน 1/3) รายละเอียดอยู่ในบทที่ 5 damage เป็น engineering proxy ไม่ใช่การทำนายการบาดเจ็บทางคลินิก จากต้นทุนจริงของ S1/S2 ประเมินงานเต็ม 56 runs ได้ 18 node-hours (base estimate) และขอ **100 node-hours** บน CPU node ของ LANTA

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

หุ่นยนต์รูปร่างมนุษย์ล้มได้เสมอเมื่อเจอแรงผลักที่แรงเกินขีดความสามารถในการทรงตัว การฝืนกู้สมดุลจนสุดทางมักจบด้วยการล้มในท่าที่แย่กว่าการล้มอย่างตั้งใจ เช่น ศีรษะหรือลำตัวกระแทกพื้นโดยไม่มีอะไรรองรับ คนจริงตัดสินใจเรื่องนี้ภายในเสี้ยววินาที ว่าจะก้าวกู้ต่อหรือจะยอมลงพื้นแล้วใช้แขนรับแรง

งานส่วนใหญ่แยกปัญหานี้เป็นหลายส่วน คือตัวทำนายการล้ม ตัวควบคุมการทรงตัว และตัวควบคุมการล้ม แล้วมีกฎสลับระหว่างกัน โครงการนี้ถามว่า policy เดียวที่เรียนจากผลของการกระทำเอง จะเรียนการตัดสินใจนี้ได้หรือไม่ และถ้าต้องล้ม จะเรียนเลือกอวัยวะรับแรงตามสิ่งที่ต้องทำต่อหลังล้มได้หรือไม่

1.2 คำถามวิจัย

1. **ควรกู้หรือควรล้ม (S2):** หุ่นรู้ได้หรือไม่ว่าแรงผลักครั้งนี้กู้ได้ ถ้ากู้ได้ให้ยืนต่อ ถ้ากู้ไม่ได้ให้ล้มแบบเสียหายน้อย โดยไม่มี label ว่าแรงไหนกู้ได้ และไม่มีตัวตัดสินใจระดับสูงแยกต่างหาก
2. **เลือกอวัยวะรับแรง (S3):** เมื่อต้องล้มแน่ และภารกิจถัดไปต้องใช้แขนหรือขา หุ่นเลือกให้อวัยวะที่ภารกิจไม่ต้องใช้รับแรงแทนได้หรือไม่

1.3 วัตถุประสงค์ของรอบนี้

- แสดงว่าโจทย์เรียนรู้ได้จริง ผ่าน gate S1 (M1) และ S2 (M2) ครบ 3 seeds
- ได้คำตอบเบื้องต้นของคำถามที่ 1
- ประเมิน compute ของงานเต็มจากต้นทุนจริง (M3) เพื่อขอโควตา LANTA
- สรุปหลักฐานสำหรับอาจารย์ที่ปรึกษา (M4)

1.4 ขอบเขต

- จำลองใน MuJoCo เท่านั้น ยังไม่มีหุ่นจริง
- damage เป็น engineering proxy: threshold รายอวัยวะอ้างอิงงานชีวกลศาสตร์ แต่อัตราสะสมและน้ำหนัก 5/2/1 เป็นค่าที่ออกแบบเอง
- S3 อยู่นอกขอบเขตรอบนี้ มีแค่แผนการทดลองและเกณฑ์
- ทุกผลรันด้วย seeds 63001–63003 และรายงานทุก seed รวมถึงที่ไม่ผ่าน

1.5 นิยามที่ใช้

คำ	ความหมายในรายงานนี้
impulse	แรงผลักไร้มิติที่กระทำต่อลำตัวในทิศแนวนอน ค่าเดียวกันใช้ทั้งเทรนและประเมิน
กู้ได้ / กู้ไม่ได้	เทียบกับ policy S1 ที่ระบุไว้ ไม่ใช่ขีดจำกัดทางฟิสิกส์สัมบูรณ์ กู้ได้ = 0.05/0.08, กู้ไม่ได้ = 0.15/0.20/0.25
ล้ม	ลำตัวต่ำกว่า 0.70 m (ร่างกาย v1.0), แกนลำตัวเอียงจน cos < 0.5 หรือส่วนที่ไม่ใช่เท้าแตะพื้น
weighted damage	5·ศีรษะ + 2·ลำตัว + แขน + ขา เฉลี่ยต่อ episode
false give-up (FGU)	สัดส่วน scenario ที่ S1 รอดแต่ S2 ล้ม บนแรงผลักที่กู้ได้
passive	ค้างท่ายืนด้วย PD (action = 0) ไม่มี policy ตอบสนอง

บทที่ 2 งานที่เกี่ยวข้อง

การเรียนรู้ล้มอย่างปลอดภัยด้วย RL มีงานก่อนหน้ามาก โครงการนี้จึงไม่อ้างว่าเป็นงานแรกที่ใช้ policy เดียวสลับจากทรงตัวเป็นล้ม จุดต่างที่ตั้งใจทดสอบคือการเลือกอวัยวะรับแรงตามผลต่อภารกิจถัดไป (S3) ซึ่งในงานที่ตรวจ (10 ก.ย. 2026) ยังไม่พบการทดสอบตรงครบเงื่อนไข

งาน	สิ่งที่ทำ	ต่างจากโครงการนี้อย่างไร
Learning to Predict Humanoid Fall (Kalyanakrishnan & Goswami, 2011)	ทำนายว่า controller ที่กำหนดจะกู้ได้หรือไม่ แล้วส่งต่อให้ fall controller	แยก predictor กับ controller; เราใช้นิยาม "กู้ได้" เทียบ controller ตามงานนี้
Unified Control Policy for Safe Falling (2017)	mixture of actor-critics เลือกส่วนที่แตะพื้น ลด impulse สูงสุด	เฉพาะช่วงล้ม ไม่รวมการตัดสินใจกู้/ล้ม
SafeFall (2025)	GRU ทำนายการล้ม + policy ลดความเสียหาย ถ่วงน้ำหนักหัวและมือสูง	ลำดับอวัยวะกำหนดผ่านน้ำหนักคงที่ ไม่ขึ้นกับภารกิจถัดไป
Guardians as You Fall (2023)	หลายโหมด + planner เลือกการเปลี่ยนผ่าน	มีสวิตช์ที่ออกแบบไว้
Arm-Assisted Fall Damage Reduction (2023)	ใช้แขนลดแรงและฟื้นตัวใน controller เดียวสำหรับช่วงฉุกเฉิน	ยังมี controller ภารกิจปกติส่งต่อ
FIRM (2025) / VIGOR (2026)	รวม fall safety กับ recovery จากตัวอย่างมนุษย์	ใช้ demonstrations ชี้นำพฤติกรรม
Embedding Classical Balance Control Principles (Poddar et al., 2026)	policy เดียวบน Unitree H1-2: ทรงตัว, ก้าวกู้, compliant falling, ลุกขึ้น	ใกล้ที่สุด; ใช้ capture-point shaping คล้ายเรา แต่ไม่มีเกณฑ์กู้/ล้มแบบ paired หรือการเลือกอวัยวะ
Discovering Self-Protective Falling (2025)	เรียนท่าค้ำสามเหลี่ยมจาก reward โดยไม่เลียนแบบมนุษย์	มีสวิตช์ walking → fall policy
Prolonging Tool Life (2025)	นำอายุใช้งานจาก FEA เข้า reward	อายุใช้งานอยู่ใน objective ตรง ไม่ใช่ผลผ่านภารกิจถัดไป

2.1 ตำแหน่งของงานนี้

- **ไม่ใหม่:** policy เดียวครอบคลุมทรงตัวและล้ม, การถ่วงน้ำหนัก damage ตามอวัยวะ, การใช้ capture point ช่วยก้าวเท้า

- **สิ่งที่งานนี้เพิ่ม:** (1) เกณฑ์ประเมินแบบ paired ที่วัดทั้งต้นทุนของการยอมล้มผิดจังหวะ (FGU) และประโยชน์ด้าน damage เทียบกับ baseline สองตัว (2) หุ่นอ้างอิงสัดส่วนมนุษย์พร้อม threshold บาดเจ็บรายส่วน (3) แผน S3 ที่ให้ความสำคัญของอวัยวะมาจากผลต่อภารกิจ พร้อม control แบบ counterfactual
- **ข้อเตือน:** reward และ curriculum เป็นสิ่งที่ออกแบบไว้ จึงอ้างว่า agent "ค้นพบเองจาก reward ภารกิจล้วน" ไม่ได้

การทบทวนนี้เป็นแบบมุ่งโจทย์ ไม่ใช่ systematic review รายละเอียดอยู่ใน `research/prior_art_review_th.md` และ `research/evidence_matrix.md`

บทที่ 3 วิธีการ

3.1 ร่างกาย v1.0

หุ่นถูกสร้างให้อ้างอิงมนุษย์จริงได้ โดยไม่จำลองกระดูกครบ 206 ชิ้น เพราะส่วนใหญ่ไม่มีผลต่อพลวัตการล้ม ทุกตัวเลขมีแหล่งอยู่ใน `configs/research/body_v1_0_female.json` และสร้าง MJCF ด้วย `scripts/build_body_v1.py`

ด้าน	ค่า / ที่มา
โครงสร้าง	SMPL 24 ท่อน, 23 ข้อต่อ × 3 แกน = 69 DoF, PD ต่อข้อต่อ (ไม่มีกล้ามเนื้อ)
คนอ้างอิง	ผู้หญิง 1.60 m / 60 kg (โมเดลได้ 1.602 m / 59.99 kg)
มวล, CoM, ความเฉื่อย	de Leva 1996 ตาราง 4 ผู้หญิง สเกล 0.922
ขนาดรูปทรง	ANSUR II ผู้หญิงสูง 1.57–1.63 m (705 คน)
ช่วงการเคลื่อนไหว	Luttgens & Hamilton 1997
แรงบิดสูงสุด	Harbo et al. 2012 ผู้หญิงอายุต่ำกว่า 30 ปี
threshold บาดเจ็บ	ศีรษะ 2.35 kN, ลำตัว 2.16 kN, เชิงกราน 5.77 kN, ไหล่ 2.52 kN, ข้อมือ 2.26 kN, เข่า 3.5 kN, เท้า 3.7 kN

ตรวจเบื้องต้น: ความสูงยอดกระดูกอก 1.297 m เทียบ ANSUR 1.307 m ยืนนิ่งได้ 100% เมื่อไม่มีแรงผลัก และขีดยืนเท้าติดพื้นตาม XCoM ค่ากลาง 0.115 ข้อจำกัดคือปิดการชนกันเองระหว่างต้นแขนกับลำตัว และไหปลาร้ากับคอ

3.2 Environment และแรงผลัก

- MuJoCo 3.13, physics 500 Hz (dt 0.002 s), control 50 Hz (dt 0.02 s), episode 10 s
- แรงผลักแนวนอนหนึ่งครั้งที่ลำตัว เกิดช่วง 1–3 s สุ่มทิศ

- ท่าเริ่มต้นสุ่มท่ายืนสบาย: ถ่ายน้ำหนักซ้าย/ขวา (เชิงกรานเลื่อน ±~5 ซม.) และระยะห่างเท้า −3 ถึง +6 ซม. อยู่ในราว ±1.5 SD ของระยะที่คนเลือกยืนเอง (16.35 ± 4.20 ซม.)
- ล้ม = ลำตัวต่ำกว่า 0.70 m หรือ cos(แกนลำตัว) < 0.5 หรือส่วนที่ไม่ใช่เท้าแตะพื้น
- S1 จบ episode เมื่อล้ม ส่วน S2 รันต่อ 2.5 s หลังล้มเพื่อเก็บ damage จากการกระแทกให้ครบ และการล้มเป็นที่สิ้นสุด (ลุกขึ้นไม่ได้โบนัสคืน)

3.3 Damage model (engineering proxy)

แบ่งร่างกายเป็น 6 ส่วน (ศีรษะ, ลำตัว, แขนซ้าย/ขวา, ขาซ้าย/ขวา) แต่ละ step แรงสัมผัสของ geom ถูกหารด้วย threshold ของส่วนนั้น แล้วสะสมเฉพาะส่วนที่เกิน:

$$\Delta d_p = 0.12 \cdot \max\left(0, \frac{F_p}{F_p^{tol}} - 1\right)^2 \cdot \Delta t, \qquad d_p \le 1$$

$$D_w = 5\,d_{head} + 2\,d_{torso} + d_{arm,L} + d_{arm,R} + d_{leg,L} + d_{leg,R}$$

threshold มาจากงานชีวกลศาสตร์ แต่อัตรา 0.12/s, เลขชี้กำลัง 2 และน้ำหนัก 5/2/1 เป็นค่าที่ตั้งเอง การล้มทั่วไปได้ D_w ราว 0.001–0.003 ต่อ episode โค้ดมีกลไก impairment ที่ลดแรงบิดตาม damage ไว้สำหรับ S3 แต่ S1/S2 ปิดไว้

3.4 Reward

S1 (ต่อ step):

$$r = \underbrace{1 + 0.5\cos\theta}_{\text{ยืนอยู่}} - 0.05\overline{a^2} - 0.05\overline{\Delta a^2} - r_{posture} + r_{CP}$$

- **posture v4 (ยืนสบาย):** เท้าเลื่อนได้ฟรี 8 ซม. (~2 SD ของคนจริง) เกินจากนั้นมีต้นทุนเพิ่มไม่สิ้นสุด แต่ reward ไม่ติดลบ, หักเมื่อ CoM ต่ำลงเกิน 3 ซม. หรือเอียงเกิน 10° ก่อนผลัก และตั้งตำแหน่งเท้าอ้างอิงใหม่ 1.5 s หลังผลัก (ก้าวกู้ฟรี แต่ขยับเท้าเรื่อยๆ มีต้นทุน)
- **capture point (น้ำหนัก 10):** โบนัสตามระยะจากจุดที่เท้าลงถึง capture point (CoM + v/ω_0; Hof 2005) แบบ Gaussian $\sigma = 10$ ซม. สำหรับการลงเท้าไม่กี่ครั้งแรกหลังผลัก

S2: reward S1 ทั้งหมด − 10,000 × การเพิ่มขึ้นของ D_w ต่อ step บวกกฎ hard-push-no-bonus ของ config สุดท้าย: ใน episode ที่แรงผลักมาจากช่วงกู้ไม่ได้ หลังเริ่มผลัก reward = −damage เท่านั้น (ไม่มีโบนัสยืน, upright, posture หรือ capture point) policy ไม่เห็นว่าเป็นแรงกลุ่มไหน ต้องอนุมานจาก state เอง

3.5 Curriculum

ขั้น	updates	แรงผลักตอนเทรน	สิ่งที่เพิ่ม
S1-A	200	≤ 0.12	posture v1 น้ำหนัก 2.0, เริ่มจากศูนย์
S1-B	300	+ 50% แบบต้องก้าว 0.13–0.25	–
S1-CP	150	เหมือน B	capture point 10, log-std −1.0
S1-R1	200	+ 50% แบบต้องก้าว 0.08–0.15	กติกายืนสบาย v4 + ท่าเริ่มสุ่ม
S1-R2 (endpoint M1)	200	+ 50% แบบต้องก้าว 0.13–0.25	–
S2 (endpoint M2)	200	กู้ได้ U(0, 0.08) / กู้ไม่ได้ U(0.15, 0.25) อย่างละครึ่ง	damage, รันต่อหลังล้ม, KL anchor, hard-no-bonus

รวม S1 = 1,050 updates (17,203,200 transitions) และ S2 = 200 updates (3,276,800 transitions) ต่อ seed

3.6 การเรียนรู้ (PPO)

พารามิเตอร์	ค่า
ไลบรารี	Stable-Baselines3 PPO, torch 2.14, CPU
เครือข่าย	actor และ critic แยกกัน ชั้นละ 256 × 2
rollout	8 envs × 2,048 steps = 16,384 transitions/update
learning rate / batch / epochs	1e-4 / 2,048 / 5
target KL / clip / GAE λ / γ	0.02 / 0.2 / 0.95 / 0.99
normalization	VecNormalize obs (clip 10) + reward

KL anchor ใน S2: เพิ่ม KL(S1 ‖ θ) เข้าไปใน PPO objective (ค่าสัมประสิทธิ์ 1.0) คิดเฉพาะบน state ที่แรงผลัก ≤ 0.08 และยังไม่ล้ม ใช้ raw observation ก่อน action เพื่อกันไม่ให้ทักษะการกู้ของ S1 หายระหว่าง fine-tune

3.7 การประเมินและเกณฑ์ (gate)

- **S1:** `scripts/stage1_survival.py` กวาด 16 ทิศ × 3 รอบ = 48 scenarios ต่อ impulse ที่ 0.08, 0.10, 0.12 และวัดท่ายืน 2 s ก่อนผลัก

- **S2:** `scripts/stage2_eval.py` ใช้ scenario คู่กัน (paired) ระหว่าง S2, S1 และ passive: กลุ่มกู้ได้ 0.05/0.08 และกลุ่มกู้ไม่ได้ 0.15/0.20/0.25 รันต่อ 2.5 s หลังล้ม ผลหลักใช้ checkpoint update 200 เท่านั้น ห้ามเลือก checkpoint อื่นภายหลัง

Gate	เกณฑ์ (ต้องผ่านทุก seed เว้นที่ระบุ)
M1	เท้าเลื่อนก่อนผลัก ⩽ 8 ซม., CoM ลด ⩽ 3 ซม., เอียง ⩽ 10°, อัตรารอดที่ 0.10 เฉลี่ย 3 seeds ⩾ 60%, อัตราก้าวที่ 0.12 > 0.08
M2	⩾ 48 episodes/กลุ่ม, รอดกู้ได้ S2 ⩾ S1 − 5 จุด, damage กู้ไม่ได้ S2 < S1 และ < passive, FGU ⩽ 10%, รายงาน head share
M3	node-hours จาก transitions จริง × runs ÷ throughput × 1.3 และมี sbatch จริง
M4	เอกสารสรุปที่ตัวเลขตรงกับไฟล์ผล

การตรวจทั้งหมดทำโดย `auto-loop/verify.ps1` (pytest + `check_goal.py`) ซึ่งอ่านไฟล์ผลที่สคริปต์ประเมินเขียนออกมา ไม่ใช่ตัวเลขที่เขียนเอง ไฟล์เกณฑ์ถูกล็อกไว้ไม่ให้แก้ระหว่างทำงาน

3.8 สภาพแวดล้อมการทดลอง

WSL2 Ubuntu 24.04 บนเครื่อง CPU 16 logical cores, RAM 8 GB, RTX 4060 (ไม่ใช้ในการเทรน), Python 3.12.3, MuJoCo 3.13.0 จาก benchmark ก่อนหน้า PPO บน CPU เร็วกว่า RTX 4060 ประมาณ 2 เท่า (median speedup CUDA 0.51–0.55x) จึงเทรนบน CPU ทั้งหมด ทุก run บันทึก manifest, source hash, updates.csv และ checkpoint ทุก 25 updates

บทที่ 4 ผลการทดลอง

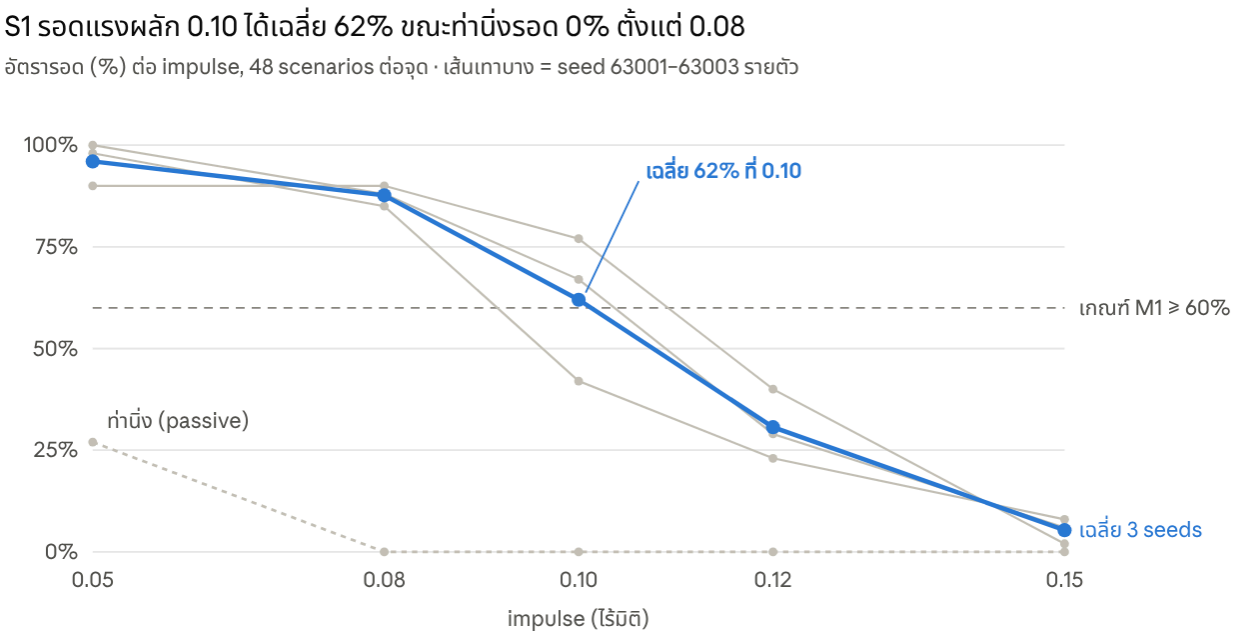
M1 และ M2 ผ่านทุกเกณฑ์ครบ 3 seeds ตาม `auto-loop/reports/goal-status.md` (verify exit 0, 10 ต.ค. 2026) ตัวเลขทุกตัวในบทนี้ดึงจากไฟล์ผลที่สคริปต์ประเมินเขียนออกมา

4.1 M1: ยืนสบายและกู้แรงผลัก (S1)

ทั้งสาม seeds ยืนสบายตามเกณฑ์ และรอดแรงผลักด้วยการก้าวเท้าตอบสนอง ไม่ใช่การยืนกางขารอไว้ก่อน run endpoint คือ `natural4_r2` update 200 (`auto-loop/milestones/m1_runs.json`)

seed	เท้าเลื่อนก่อนผลัก (ซม.)	CoM ลด (ซม.)	ลำตัวเอียง (°)	รอดที่ 0.10	ผู้รอดที่ก้าว 0.08 → 0.12
63001	4.8	−0.1	2.2	42%	61% → 82%
63002	7.3	−0.1	5.0	67%	50% → 93%
63003	4.8	0.2	3.7	77%	63% → 79%

seed	เท้าเลื่อนก่อนผลัก (ซม.)	CoM ลด (ซม.)	ลำตัวเอียง (°)	รอดที่ 0.10	ผู้รอดที่ก้าว 0.08 → 0.12
เกณฑ์	⩽ 8	⩽ 3	⩽ 10	เฉลี่ย ⩾ 60% (ได้ **62%**)	เพิ่มขึ้น



ภาพ 4.1 · results/stage1_v1_0/seed_*/..._natural4_r2/survival_curve.json และตารางท่านิ่ง v1.0 ใน Curriculum_Plan_TH.md

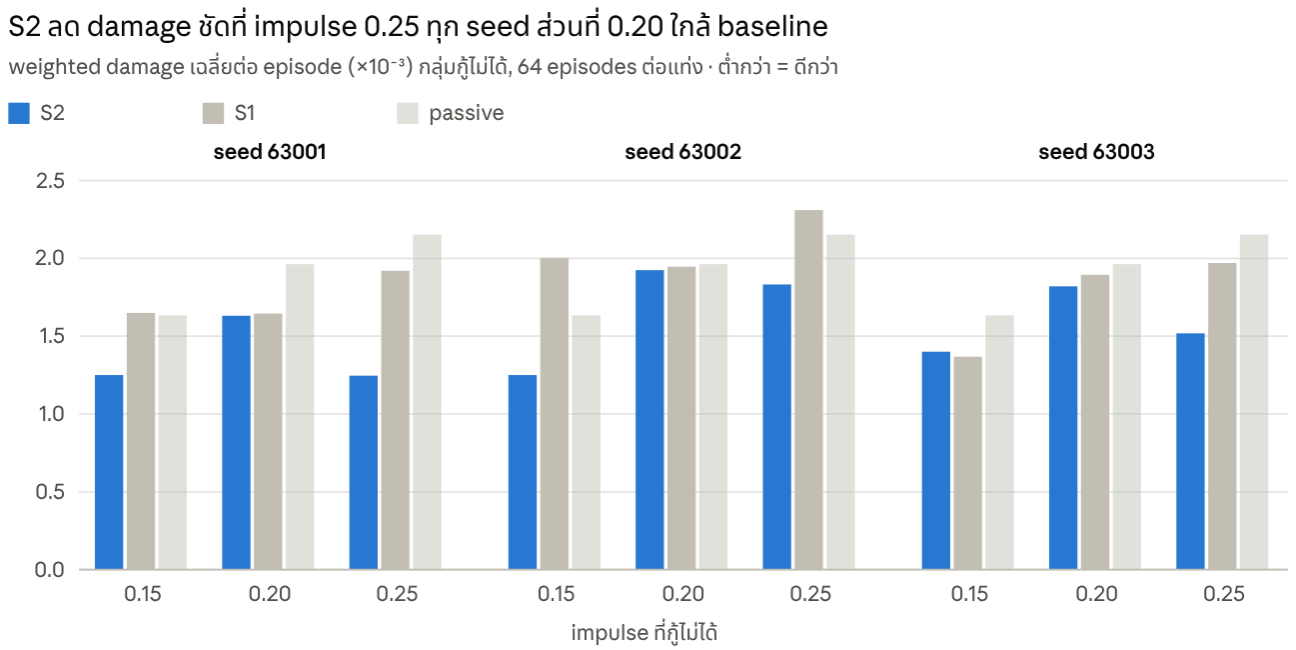
- จุดรอด 50% อยู่ราว impulse 0.10–0.11 (Δv ~0.44 m/s) ใกล้ขีดยืนเท้าติดพื้นตามทฤษฎี XCoM (0.115) ขณะที่ท่านิ่งรอด 0% ตั้งแต่ 0.08
- อัตรารอดต่างกันมากระหว่าง seed (0.10: 42–77%) ค่าเฉลี่ย 62% ผ่านเกณฑ์ 60% แบบเฉียดฉิว
- posture v4 แลกอัตรารอดกับท่ายืนที่เป็นธรรมชาติ: รอบก่อน (v3) รอดที่ 0.10 ได้ 72% แต่เท้าเลื่อน 10–12 ซม. เกินเกณฑ์ 2 seeds

4.2 M2: เลือกกู้หรือล้ม (S2)

config สุดท้าย `s2_dmg10000_anc1.0_objective_hardnobonus` ผ่านทุกเกณฑ์ครบ 3 seeds ที่ update 200 ประเมิน 192 episodes ต่อกลุ่มต่อ seed (job `20261010-120730-s2hardnob` ใช้ 4:11:18 ชม.) damage หน่วย 10^{-3}

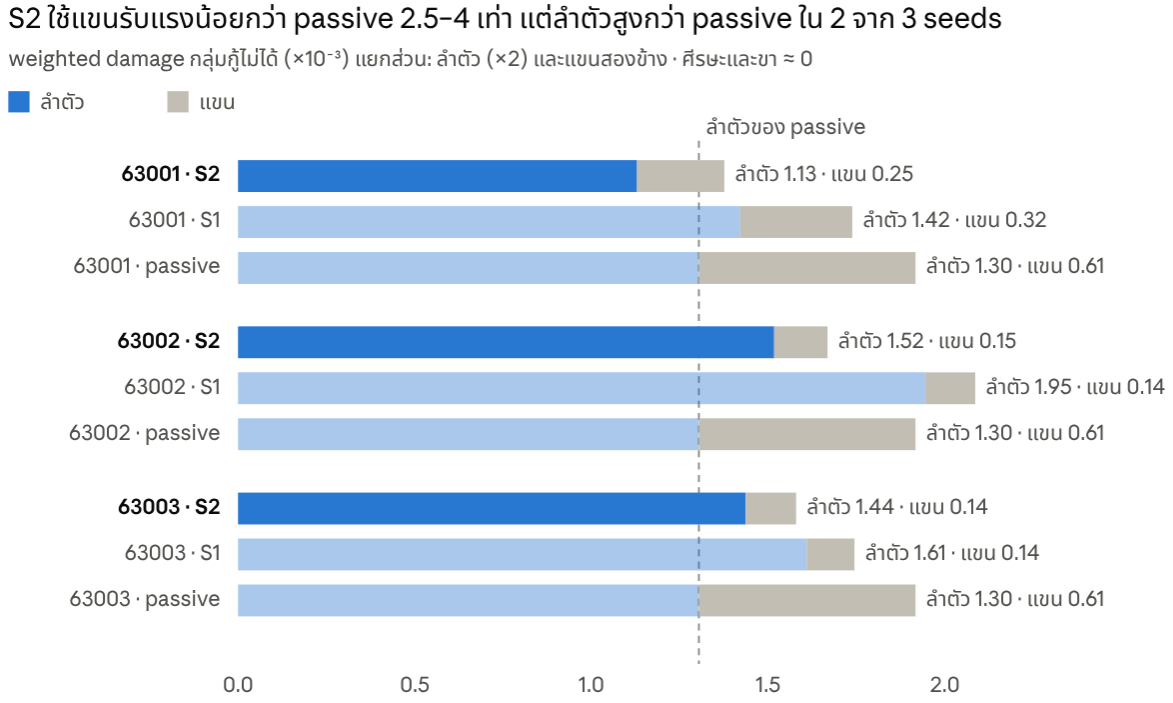
seed	รอดกู้ได้ S2 / S1	damage กู้ไม่ได้ S2 / S1 / passive	S2 ต่ำกว่า S1 / passive	FGU	head share
63001	91.7% / 90.6%	1.376 / 1.738 / 1.917	21% / 28%	7.47%	0%
63002	96.9% / 96.9%	1.669 / 2.086 / 1.917	20% / 13%	2.69%	0.00006%
63003	88.5% / 85.4%	1.579 / 1.744 / 1.917	9% / 18%	**9.76%**	0.0201%
เกณฑ์	⩾ S1 − 5 จุด	< S1 และ < passive		⩽ 10%	รายงาน

บนแรงที่กู้ได้ passive รอดเพียง 16.1% (0.05: 30.2%, 0.08: 2.1%) ส่วน S2 และ S1 รอด 85–100% บนแรงที่กู้ไม่ได้ S2 รอด 1.6–6.2% (S1 0.5–4.2%) ค่า damage เฉลี่ยรวมผู้รอดด้วยตามนิยามเดิม



ภาพ 4.2 · per_impulse ใน s2_eval.json ของ config hardnobonus update 200 ทั้ง 3 seeds

- S2 ลด damage ได้ชัดที่สุดที่ impulse 0.25 (ทุก seed) และ 0.15 (63001, 63002) ส่วนที่ 0.20 ใกล้ S1 และ passive ในสอง seeds
- seed 63003 ที่ 0.15 S2 สูงกว่า S1 เล็กน้อย (1.400 เทียบ 1.368) ผลรวมจึงต่ำกว่า S1 เพียง 9%



ภาพ 4.3 · per_part_damage_mean_unrecoverable ใน s2_eval.json (192 episodes ต่อ policy ต่อ seed); passive ใช้ค่าเดียวกันทุก seed

- damage เกือบทั้งหมดมาจากลำตัว ส่วนขาไม่เคยเกิน threshold และศีรษะแทบไม่เคยถึง threshold

- **ข้อสังเกตสำคัญ:** ลำตัวของ S2 ยังรับ damage มากกว่า passive ใน 2 จาก 3 seeds ส่วนที่ S2 ชนะ passive มาจากแขนที่เสียหายน้อยกว่า 2.5–4 เท่า passive ล้มแบบไม่ต้านและใช้แขนรับแรง ส่วน S2 ยังลงพื้นด้วยลำตัวเป็นหลัก จึงยังไม่ได้เรียน "ท่าล้มที่ดี" แบบที่คนอาจคาดหวัง

4.3 Diagnostic ที่ไม่ใช่ gate

เวลาตั้งแต่ถูกผลักจนล้ม (`scripts/stage2_fall_timing.py` , config dmg10000/dmg20000 ก่อน config สุดท้าย, 48 hard episodes): S2 ล้มช้ากว่า passive เพียง ~0.1 s (median) และไม่ช้ากว่า S1 (paired S2−S1 −0.15 ถึง +0.02 s) เวลาล้มไม่ทำนาย damage ($|r| \leqslant 0.10$, $n \approx 144$) ส่วนต่าง damage จึงมาจากท่าลงพื้น ไม่ใช่เวลาล้ม ยังไม่ได้วัด diagnostic นี้กับ config สุดท้าย

ประเมินซ้ำบน scenario ชุดใหม่ (reset seeds 19000–19095, config joint KL ก่อนหน้า ไม่ใช่ config สุดท้าย): bootstrap CI95 ของ damage S2−baseline ต่ำกว่าศูนย์ชัดเจนเฉพาะ seed 63003 ส่วน 63001/63002 คร่อมศูนย์ ผลนี้เตือนว่าความต่าง damage ระดับ 10–20% ต้องใช้ episodes มากจึงแยกจาก noise ได้ และยังไม่มี CI ของ config สุดท้าย

4.4 การตรวจสอบอิสระหลังลูปจบ

- `verify.ps1` exit 0 เมื่อรันซ้ำ (pytest ผ่านทั้งหมด, M1–M4 PASS)
- `verify.ps1` , `GOAL.md` , `check_goal.py` ไม่ถูกแก้ tests มีแต่เพิ่ม
- มี run เดียวต่อ seed ต่อ config ไม่มีการเลือก run หรือ checkpoint ที่ดีที่สุด
- ตัวเลข M2 ในเอกสารตรงกับ `s2_eval.json` ทุก seed

บทที่ 5 อภิปรายและข้อจำกัด

ผล M2 สนับสนุนว่า policy เดียวประนีประนอมระหว่างการกู้และการลด damage ได้ แต่เป็นหลักฐานเบื้องต้นที่ไวต่อชุดประเมิน ไม่ใช่ผลที่มั่นคง บทนี้เปิดเผยทุกสิ่งที่ผู้พิจารณาควรรู้ก่อนตีความผล

5.1 ประวัติการลอง config ของ S2

ก่อนผ่าน M2 ได้ลอง config ทั้งหมด 10 ชุด (ไม่รวม control) ทุกชุดใช้ seed 63001–63003 และรายงาน endpoint update 200 ตามกฎที่ตั้งไว้ก่อนรัน ไม่มีการเลือก checkpoint ภายหลัง

#	config	สิ่งที่เปลี่ยน	episodes ต่อกลุ่ม	seeds ผ่าน	สาเหตุหลักที่ไม่ผ่าน
1	`s2_dmg10000`	ฐาน: damage ×10,000, รันต่อหลังล้ม	48	0/3	FGU 12–30%, รอดกู้ได้ตก 27 จุด
2	`_cw25`	critic warm-up 25 updates	48	0/3	FGU 13–33%
3	`_lr3e-5`	learning rate 3e-5	48	1/3	FGU ถึง 65%
–	`s2_dmg0` (control)	ไม่มีโทษ damage	48	–	FGU ก็ยังแกว่ง 2–29% → ต้นเหตุไม่ใช่ damage term

#	config	สิ่งที่เปลี่ยน	episodes ต่อกลุ่ม	seeds ผ่าน	สาเหตุหลักที่ไม่ผ่าน
4	`push_0.12`	ช่วงกู้ได้ U(0, 0.12)	48	0/3	FGU 15–20%
5	`_anc1.0`	KL anchor ไป S1 (แยก phase, 5 epochs)	48	0/3	damage ไม่ลด (anchor ล็อกพฤติกรรม)
6	`_anc1.0_ae1`	anchor 1 epoch	48	2/3	63002 damage > passive 9%
7	wide + ae1	รวม 4 + 6	48	0/3	FGU และ damage
8	`_objective`	KL ใน PPO objective	48	2/3	63002 damage > passive 4.7%
9	`s2_dmg20000`	damage ×20,000	48	2/3	63002 damage > S1 และ passive
10	`_hardnobonus`	reward บนแรงกู้ไม่ได้ = −damage เท่านั้น	**192**	**3/3**	–

ใช้ compute กับ S2 รวมราว 40 ชั่วโมง WSL รวม control และ diagnostic (~2–4.5 ชม. ต่อ config ทั้ง 3 seeds) การปรับ recipe ซ้ำบน scenario ชุดเดิมหลายรอบเป็นการเปรียบเทียบหลายครั้ง (multiple comparisons) จึงมีโอกาสที่ผลผ่านสูงเกินจริง

5.2 ผลผ่านขึ้นกับขนาดชุดประเมิน

ข้อนี้สำคัญที่สุดในบท: config 10 ผ่าน 3/3 เมื่อประเมิน 192 episodes แต่ถ้าใช้ชุด 48 episodes เดิมที่ใช้ตัดสิน config 1–9 จะ**ผ่านเพียง 1/3** (`s2_eval_n48.json`):

seed	รอดกู้ได้ S2 / S1	damage S2 / S1 / passive (10^{-3})	FGU	ผลที่ 48	ผลที่ 192
63001	91.7% / 87.5%	1.230 / 1.867 / 1.607	9.5%	ผ่าน	ผ่าน
63002	95.8% / 95.8%	1.737 / 1.899 / 1.607	4.3%	ไม่ผ่าน (damage > passive)	ผ่าน
63003	81.2% / 95.8%	1.564 / 1.427 / 1.607	19.6%	ไม่ผ่าน (รอด, FGU, damage > S1)	ผ่าน

- การเปลี่ยนเป็น 192 episodes ถูกกำหนดและบันทึกไว้**ก่อนรัน** config 10 (เหตุผล: verdict ต่อ seed พลิกไปมาระหว่าง checkpoint ติดกันที่ 48 episodes) และ GOAL กำหนดขั้นต่ำ ⩾ 48 จึงไม่ผิดกติกา แต่ config 1–9 ไม่ได้ถูกประเมินซ้ำที่ 192 จึงเทียบกันไม่เท่าเทียม
- ค่า S1 ของ seed เดียวกันก็ต่างกันมากระหว่างสองชุด (63003 รอด 95.8% ที่ 48 แต่ 85.4% ที่ 192) แสดงว่า noise ของการประเมินใหญ่พอๆ กับขนาด effect ที่วัด
- trend checkpoint (update 50/100/150 ที่ 48 episodes) ผ่านบ้างไม่ผ่านบ้างทุก seed ไม่มีแนวโน้มชัด
- **ข้อสรุปที่ปลอดภัย:** "ผ่าน gate M2 ตามเกณฑ์ที่กำหนดไว้ก่อนรัน แต่ความต่างจาก baseline ยังไม่มั่นคงข้ามชุดประเมิน" ต้องประเมินบน scenario ชุดใหม่ที่ไม่เคยใช้ปรับ recipe พร้อม CI ก่อนอ้าง superiority

5.3 Reward ของ config ที่ผ่านถูกออกแบบเพิ่ม

กฎ hard-push-no-bonus ตัดโบนัสยืนออกเมื่อแรงผลักมาจากช่วงกู้ไม่ได้ ซึ่งใช้ข้อมูล class ของแรงผลักที่ผู้ออกแบบรู้ในการคำนวณ reward (privileged information) แม้ policy จะมองไม่เห็น class ก็ตาม เหตุผลคือ reward เดิมให้ ~1.5/step ตราบที่ยังไม่ล้ม การเอามือยันพื้นนับเป็นการล้ม จึงเสียโบนัส ~7.5 ซึ่งใกล้เคียงส่วนต่าง damage ที่ต้องการ reward เดิมจึงจ่ายให้ "ไม่ใช้แขนรับแรง"

ผลที่ตามมาคือ คำตอบของคำถาม S2 ในรอบนี้คือ "policy เรียนได้เมื่อ reward บอกว่าแรงไหนไม่คุ้มที่จะฝืน" ไม่ใช่ "policy ค้นพบเองจากต้นทุน damage ล้วน" การเขียนผลต้องระบุข้อนี้ชัดเจน

5.4 ความหมายของ "เลือกล้ม"

- บนแรงที่กู้ไม่ได้ S1 ก็ล้มเกือบทุกครั้งเช่นกัน (รอด 0.5–4.2%) ความต่างจึงอยู่ที่ "ล้มอย่างไร" มากกว่า "ตัดสินใจล้มหรือไม่"
- ยังแยกไม่ได้ว่า policy คาดการณ์ความกู้ได้จริงมากน้อยเพียงใด ยังไม่มี probe หรือการวัดเวลาตัดสินใจหลังถูกผลัก
- ท่าลงพื้นของ S2 ยังใช้ลำตัวเป็นหลัก และลำตัวเสียหายมากกว่า passive ใน 2/3 seeds (ภาพ 4.3)

5.5 ข้อจำกัดอื่น

ข้อจำกัด	ผลกระทบ	วิธีแก้ที่เสนอ
damage เป็น engineering proxy	ตีความเป็นการบาดเจ็บจริงไม่ได้	เทียบกับข้อมูลการล้มของมนุษย์หรือ dummy ที่มีเซนเซอร์แรง
เกณฑ์ผ่านแบบเฉียดขอบ (FGU 63003 9.76%, M1 เฉลี่ย 62%)	อาจไม่ผ่านเมื่อเทรนซ้ำ	เพิ่ม seeds เป็น 5
baseline มีแค่ passive และ S1 ของเรา	ยังไม่รู้ว่าดีกว่าวิธีที่มีอยู่แล้วหรือไม่	เพิ่ม baseline แบบ predictor + fall controller หรือสคริปต์ท่าล้ม
หุ่นเป็น PD ไม่มีกล้ามเนื้อ ปิดการชนกันเองบางคู่	ท่าล้มบางแบบอาจไม่สมจริง	ทดสอบกับโมเดลหุ่นจริง (เช่น Unitree G1/H1)
นิยาม "กู้ได้" ผูกกับ S1 และช่วง impulse ที่เลือก	ช่วง 0.08–0.15 ไม่ถูกประเมิน	ประเมินทั้งช่วงต่อเนื่อง
ยังไม่มีวิดีโอของ policy S2 สุดท้าย (GOAL กำหนดให้มี)	ยังไม่ได้ตรวจ reward hacking ด้วยตา	render ด้วย `scripts/watch_policy.py` ก่อนส่ง
งานส่วนใหญ่ทำโดยลูป AI อัตโนมัติ (Claude + Codex)	ต้องมีคนตรวจทาน	ตรวจทานอิสระแล้ว (4.4) แต่ควรให้อาจารย์ตรวจโค้ด reward อีกครั้ง

.

บทที่ 6 แผน S3 และคำขอทรัพยากร LANTA

ขอ **100 node-hours บน CPU node ของ LANTA** เพื่อทำ S3 และยืนยัน S1/S2 ด้วย seeds ที่มากขึ้น ค่าพื้นฐานคำนวณได้ 18 node-hours ส่วนที่เหลือเผื่อความไม่แน่นอนของ throughput และ S3

6.1 แผน S3: เลือกอวัยวะรับแรง

แนวคิด: เมื่อต้องล้มแน่ หุ่นลงพื้นได้หลายแบบ หลังล้มมีภารกิจถัดไปที่ต้องใช้แขนหรือขา agent ควรยอมเจ็บส่วนที่ภารกิจไม่ต้องใช้ กลไก impairment ที่มีอยู่แล้วลดแรงบิดสูงสุดตาม damage ของอวัยวะนั้น จึงเกิดแรงจูงใจจากฟิสิกส์เองโดยไม่ต้องใส่ reward พิเศษ

- **cue:** one-hot ARM/LEG ใน observation แสดงก่อนหรือพร้อมแรงผลัก
- **แรงผลัก:** เฉพาะแบบกู้ไม่ได้ บังคับให้ล้ม
- **ภารกิจถัดไป (จากท่านอน/นั่ง):** ARM = เอามือแตะเป้าและค้าง, LEG = เอาเท้าแตะเป้าและค้าง
- **reward:** ช่วงล้ม $-\lambda \times$ weighted damage, ช่วงภารกิจ $+R$ เมื่อสำเร็จและ shaping ตามระยะ
- **ต้อง calibrate ก่อน:** damage ของแขน/ขาต้องทำให้อัตราสำเร็จของภารกิจที่ใช้อวัยวะนั้นลดลงจริง ไม่งั้น agent ไม่มีเหตุผลต้องเลือก

ตัวชี้วัดหลัก (counterfactual): ใช้แรงผลักและ seed เดียวกัน เปลี่ยนแค่ cue

```
SSI = \left(D_{leg}^{ARM} - D_{leg}^{LEG}\right) + \left(D_{arm}^{LEG} - D_{arm}^{ARM}\right)
```

SSI > 0 แปลว่า agent ปกป้องอวัยวะที่ภารกิจถัดไปต้องใช้

Control	สิ่งที่ตรวจ	SSI ที่คาด
ไม่มี cue	policy ไม่รู้ภารกิจถัดไป	~0
ปิด impairment (สำคัญที่สุด)	การเลือกเกิดจากผลของ damage จริง	~0
cue สลับมั่ว	policy ใช้ cue จริง	~0
λ sweep	กราฟ trade-off ความสำเร็จกับ damage รวม	รายงานทั้งเส้น

Gate S3 (3 seeds): SSI > 0 สม่ำเสมอทุก seed, สูงกว่า control ไม่มี cue และปิด impairment ชัดเจน และอัตราสำเร็จของภารกิจถัดไปสูงกว่ากรณีไม่มี cue บทเรียนจาก S2 คือต้องกำหนดชุดประเมินและขนาด ($\geq$ 192 episodes) ไว้ตั้งแต่ต้น และกัน scenario สำหรับประเมินสุดท้ายไว้ไม่ใช้ปรับ recipe

ความเสี่ยง: ท่าลงพื้นบางแบบอาจทำไม่ได้ทางฟิสิกส์ (แรงบิดไหล่จำกัด), ภารกิจอาจง่ายหรือยากเกินจน damage ไม่มีผล, และจากภาพ 4.3 ปัจจุบัน S2 ลงพื้นด้วยลำตัวแทบทั้งหมด แขนและขารับ damage น้อยมาก จึงอาจไม่มี damage แขน/ขามากพอให้สลับได้ ต้องตรวจในขั้น calibrate ก่อน

6.2 การประเมิน compute

ทุก run ใช้ 8 envs × 2,048 steps = 16,384 transitions/update throughput WSL ต่อ run: S1 1,931–1,948 t/s, S2 1,180–1,243 t/s

ส่วน	config	seeds	transitions ต่อ run	รวม transitions	ที่มา
S1 ทำซ้ำเพื่อยืนยัน	1	5	17,203,200	86,016,000	วัดจริง
S2 ablation / threshold / LR	8	3	3,276,800	78,643,200	วัดจริง
S3 หลัก + controls + λ sweep	9	3	13,107,200	353,894,400	สมมติฐาน 800 updates
evaluation overhead 10%				51,855,360	
รวม (56 runs)				**570,408,960**	

```
\text{node-hours} = \frac{570{,}408{,}960 \times 1.3}{12{,}000 \times 3{,}600} = 17.17 \rightarrow 18
```

- throughput 12,000 t/s ต่อ node มาจากสมมติฐาน 10 runs × 1,200 t/s บน CPU node 128 cores **ยังไม่ได้วัดบน LANTA** ถ้าได้ครึ่งเดียวจะเป็น ~35 node-hours
- เสนอขอ **100 node-hours** ครอบคลุม benchmark, การแก้ recipe ของ S3 (คาดจาก S2 ที่ต้องลอง 10 config) และการทำซ้ำ seed ที่ไม่ผ่าน
- ควรขอ CPU node เพราะ benchmark P3-3D แสดงว่า PPO งานนี้บน CPU เร็วกว่า GPU ~2 เท่า

6.3 Slurm scripts ที่เตรียมไว้ (ยังไม่ submit)

ไฟล์	งาน
`scripts/lanta/bench_throughput.sbatch`	วัด throughput ต่อ node ด้วย K = 10 runs, 20 updates
`scripts/lanta/s1_recipe.sbatch`	รัน S1 recipe หลาย seeds และ resume ได้
`scripts/lanta/s2_pilot.sbatch`	fine-tune/eval S2 หลาย seeds รองรับ anchor

ก่อน submit ต้องใส่ `<project-account>` , ตรวจ partition/time limit และติดตั้ง `~/venvs/rl_fall` บน LANTA

6.4 ลำดับงานหลังได้โควตา

1. วัด throughput จริงด้วย `bench_throughput.sbatch` แล้วแก้ estimate เป็น revision ใหม่
2. ยืนยัน S2 ด้วย 5 seeds และ scenario ชุดใหม่พร้อม CI
3. calibrate ภารกิจ ARM/LEG กับระดับ damage
4. S3 หลัก + controls + λ sweep

บทที่ 7 สรุป

โจทย์ "ควรกู้หรือควรล้ม" เรียนรู้ได้ด้วย policy เดียวบนหุ่นอ้างอิงมนุษย์ แต่ผลยังเป็นหลักฐานเบื้องต้นที่พอสำหรับขอ compute ไม่ใช่ผลงานพร้อมตีพิมพ์

คำถาม	คำตอบจากกรอบนี้	ความมั่นใจ
หุ่นยืนสบายและก้าวกู้ได้ไหม (S1)	ได้ 3/3 seeds จุดรอด 50% ใกล้ขีดทฤษฎี	สูง แต่ผ่านเกณฑ์เฉียดขอบ
policy เดียวลด damage โดยไม่เสียการกู้ได้ไหม (S2)	ได้ตาม gate ที่ 192 episodes; damage ต่ำกว่า S1 9–21%	ปานกลางถึงต่ำ: ไวต่อชุดประเมิน และต้องใช้ reward ที่รู้ class แรงผลัก
หุ่นเลือกอวัยวะรับแรงตามภารกิจถัดไปได้ไหม (S3)	ยังไม่ได้ทดลอง	–
ต้องใช้ compute เท่าไร	18 node-hours (base), ขอ 100	ปานกลาง: throughput LANTA ยังไม่ได้วัด

7.1 สิ่งที่ควรทำก่อนส่งรายงานนี้

- [ ] render วิดีโอ/ภาพท่าทางของ policy S2 สุดท้ายทั้ง 3 seeds ที่แรงกู้ได้และกู้ไม่ได้ (GOAL M4 กำหนด)
- [ ] รัน `bench_throughput.sbatch` บน LANTA ถ้าเข้าถึงได้
- [ ] ใส่ตาราง 5.1 และ 5.2 ลงใน `planning/Evidence_Summary_TH.md` ด้วย (ฉบับย่อยังไม่ได้เปิดเผยสองเรื่องนี้)
- [ ] ประเมิน config สุดท้ายบน scenario ชุดใหม่ (reset seeds ที่ไม่เคยใช้) พร้อม bootstrap CI ใช้เวลา WSL ประมาณ 1 ชั่วโมง

ภาคผนวก ก: ไฟล์หลักฐาน

หัวข้อ	path (นับจาก root โปรเจกต์; results อยู่ใน `~/RL_Fall` บน WSL)
เป้าหมายและเกณฑ์	`auto-loop/GOAL.md`
บันทึกการทดลองทุกครั้ง	`planning/Curriculum_Plan_TH.md`
M1 runs	`auto-loop/milestones/m1_runs.json` → `results/stage1_v1_0/seed_*/push_0.12_hard0.5_posture1.0_natural4_r2/`

หัวข้อ	path (นับจาก root โปรเจกต์; results อยู่ใน `~/RL_Fall` บน WSL)
M2 eval	`auto-loop/milestones/m2_summary.json` → `results/stage2/seed_*/push_0.08_hard0.5_posture1.0_s2_dmg10000_anc1.0_objective_hardnobonus/3d_*/s2_eval.json`
M2 ที่ 48 episodes	โฟลเดอร์เดียวกัน `s2_eval_n48.json`, trend `s2_eval_update_{50,100,150}.json`
M3 compute	`planning/LANTA_Compute_Estimate_TH.md`, `auto-loop/milestones/m3_compute.json`
เอกสารสรุปฉบับย่อ (M4)	`planning/Evidence_Summary_TH.md`
diagnostics	`results/stage2_diagnostics/`
เวลาและสถานะ job	`auto-loop/reports/jobs.md`
ผล verify ล่าสุด	`auto-loop/reports/goal-status.md`
prior art	`research/prior_art_review_th.md`, `research/evidence_matrix.md`

ภาคผนวก ข: วิธีทำซ้ำ

```
# ติดตั้ง (Ubuntu 24.04 / WSL2)
bash scripts/setup_linux.sh
# S1 ทั้ง recipe (resume ได้)
bash scripts/run_stage1_v1_recipe.sh 63001 63002 63003
# S2 config สุดท้าย + ประเมิน 192 episodes
bash scripts/run_stage2_hardnobonus.sh
# ตรวจ gate ทั้งหมด (จาก Windows)
pwsh -File auto-loop/verify.ps1
```

เวลาโดยประมาณบนเครื่อง 16 logical cores: S1 ~4 ชม. ต่อ seed, S2 ~1.4 ชม. ต่อ seed รวมประเมิน ทุก run บันทึก git commit, source hash และ checkpoint hash ไว้ใน `manifest.json`